計算幾何詳解

33-E01 精確方向與退化

推導

相對ab的o_1,o_2在(i)都是0，相對cd的o_3,o_4也都是0；c在ab的box內，交集為[(2,0),(6,0)]。(ii)$o_1=-6$、$o_2=12$，但相對cd的$o_3=21$、$o_4=3$同正，故不交：只跨ab延長線不夠。(iii)四方向均0，單點c=(6,0)在ab內，因此相交於b。

32-bit座標的差可超過signed32範圍，乘積更大。例如a=(0,0),b=(100000,0),c=(0,100000)，方向10^10已超過2^31−1，甚至三點座標本身都不大。必先擴寬，並檢查全域界$8B^2$是否符合所用型別；完整signed32輸入可能需要比signed64更寬的中間值。

結論與檢核

結果依序為重疊、分離、端點相交。精確符號及box兩者缺一不可；不以固定ε替代整數判定。

33-E02 掃描線不能只測插入

推導

x=0先插a再c，次序a,c，兩者不交（a到y=2在x=4，超過c終點3）。x=2插b，次序a,c,b，b與c不交（b到y=2在x=22/3>3）。x=3刪c前，a高度3/2，b高度21/4；新鄰居a,b需測試。解x/2=15/2−3x/4，得x=6、y=3，位於兩段內，回true。

加入v時，x=2先插b，再做垂直range query。a高度1、c高度2、b高度6，找高度≥3的第一段是b但6>4，所以v不與任何active非垂直段相交。也沒有同x其他垂直段；仍在x=3由a,b得到true。

結論與檢核

垂直段不能只和任意一個鄰居比較，更不能以其中點3.5當單一key；必測整個[3,4]高度範圍。只測插入會漏掉c刪除所揭露的a,b。

33-E03 不可把判定器當列舉器

推導

取a:(0,0)→(4,4)、b:(0,4)→(4,0)、c:(0,5)→(4,5)。x<2時高度次序a,b,c；a,b在(2,2)相交後，x>2應為b,a,c。若發現交集只列印、不在交點交換status，樹內仍a,b,c，後續above/below便不代表幾何鄰居。

列舉版本須把預測的鄰居交點加入event queue，依交點時間處理，交換或重排相交群，更新新鄰居交點；還要去重、排除已過事件及處理多段同點、垂直／重疊等退化。不能直接沿用只有端點事件的證明與成本。

計算幾何詳解 （續）

結論與檢核

判定版本可以提前停，是其status正確性的關鍵前提。此反例指出不變量在x=2後即失效，並不宣稱每個三段例一定立刻漏列某個額外交點。

33-E04 凸包與邊界約定

推導

起點O。G與A同射線，保留較遠A；F與C同射線，保留較遠C。極角次序O,A,B,C,D,E。初始O,A,B中orient(O,A,B)=6。加入C時orient(A,B,C)=−8，刪B；orient(O,A,C)=24>0，得O,A,C。加入D時orient(A,C,D)=16，保留。加入E時orient(C,D,E)=−6，刪D；orient(A,C,E)=24，得O,A,C,E。

全部共線例如(0,0),(1,1),(2,2),(1,1)，先去重，只回(0,0),(2,2)，不把中間(1,1)列為極端頂點。單一唯一座標則只回一點。

結論與檢核

矩形凸包O,A,C,E，面積24。B,D,F在內部，G在下邊中間；這些點被包含，但不屬本題輸出清單。

33-E05 Pop 的代數證明

推導

r,u線性獨立，D>0，故t唯一可寫αr+βu。用叉積解得α=orient(O,t,u)/D≥0、β=orient(O,r,t)/D≥0，因t方向在兩射線之間。展開(t−r)×(u−r)：((α−1)r+βu)×(u−r)=(α−1)D+βD=(α+β−1)D。

若方向≤0，D正使α+β≤1；t是O,r,u的非負係數和為1的凸組合。t與三者不同，不能是此四點凸包的極端點。等號時在ru邊上，嚴格小於時在三角形內或O相鄰邊上；均可刪t而不改凸包。

結論與檢核

幾何次序與總跨度<π是α,β非負的前提，不能對任意三個方向只由非左轉就宣稱t在三角形內。Graham的起點與極角順序提供了這些條件。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

計算幾何詳解 （續）

33-E06 同 X 的最近點對

推導

按rank左組ID1–4，y=0,10,20,30，最短10；右組ID5–8，y=31,40,50,60，最短9（31到40）。δ=9，ℓ=0，所有點都在strip。y順序不變；跨組ID4與5的y差1、距離1，為全域最佳。

若用x<0分流，左組為空、右組仍含全部8點，與先前X按rank分割不一致；若遞迴也照此規則，問題大小不減，不能使用平衡T(n)=2T(n/2)+O(n)。同x應按唯一ID歸屬分組。

結論與檢核

七點裝箱只要求同組距離≥δ、左組x≤ℓ、右組x≥ℓ，允許等號。分割線上的點按組分別計帳；不要求嚴格幾何分離。

33-E07 時間與位元成本

推導

暴力需n(n−1)/2次距離比較，Θ(n^2)。每層排序Y得到T(n)=2T(n/2)+Θ(n log n)=Θ(n $\log^2 n$)的實作上界／相應排序成本；預排序後各層穩定分流與常數鄰居比較只O(n)，總O(n log n)。

平方根在非負數上嚴格遞增，故比較$(dx)^2+(dy)^2$等價於比較距離。整數平方和與orientation均可用精確整數，但b位元座標的差約b+1位，平方／叉積約2b量級位元；乘法成本依M(b)，比較依O(b)。因此幾何操作數界須再配合位元成本，不是任意精度也有固定機器時間。

結論與檢核

分治省的是候選配對與排序次數；平方距離省去不必要的根號誤差。兩者都沒有消除大整數的實際位元成本。

計算幾何詳解 （續）

原創驗算程式附錄: geometry_lab.py

```
"""Exact integer/rational models; no floating epsilon is used for topology."""
from functools import cmp_to_key
from fractions import Fraction

def orient(a,b,c):return (b[0]-a[0])*(c[1]-a[1])-(b[1]-a[1])*(c[0]-a[0])
def distance2(a,b):return (a[0]-b[0])**2+(a[1]-b[1])**2
def on(a,b,p):return orient(a,b,p)==0 and min(a[0],b[0])<=p[0]<=max(a[0],b[0]) and
min(a[1],b[1])<=p[1]<=max(a[1],b[1])
def intersect(a,b,c,d):
    o1,o2,o3,o4=orient(a,b,c),orient(a,b,d),orient(c,d,a),orient(c,d,b)
    sign=lambda n:(n>0)-(n<0)
    return (sign(o1)*sign(o2)<0 and sign(o3)*sign(o4)<0) or on(a,b,c) or on(a,b,d) or
on(c,d,a) or on(c,d,b)

def graham(points):
    points=sorted(set(points))
    if len(points)<3:return points,[]
    p0=min(points,key=lambda p:(p[1],p[0]))
    def compare(a,b):
        o=orient(p0,a,b)
        if o:return -1 if o>0 else 1
        return (distance2(p0,a)>distance2(p0,b))-(distance2(p0,a)<distance2(p0,b))
    candidates=sorted([p for p in points if p!=p0],key=cmp_to_key(compare));rays=[]
    for p in candidates:
        if rays and orient(p0,rays[-1],p)==0:rays[-1]=p
        else:rays.append(p)
    stack=[];trace=[]
    for p in [p0]+rays:
        popped=[]
        while len(stack)>1 and orient(stack[-2],stack[-1],p)
<=0:popped.append((stack[-1],orient(stack[-2],stack[-1],p)));stack.pop()
        stack.append(p);trace.append((p,popped,list(stack)))
    return stack,trace

def jarvis(points):
    points=sorted(set(points))
    if len(points)<2:return points
    start=min(points);p=start;hull=[]
```

計算幾何詳解 （續）

```
    while True:
        hull.append(p);q=next(x for x in points if x!=p)
        for r in points:
            o=orient(p,q,r)
            if o<0 or (o==0 and distance2(p,r)>distance2(p,q)):q=r
        p=q
        if p==start:return hull

def closest(points):
    # IDs distinguish coincident coordinates; rank marks give exact stable splits.
    x=sorted([(p[0],p[1],i) for i,p in enumerate(points)])
    if len(x)<2:return None
    rank={p[2]:i for i,p in enumerate(x)}
    y=sorted(x,key=lambda p:(p[1],p[0],p[2]))
    def solve(x,y):
        if len(x)<=3:return min((distance2(a,b),(a,b)) for i,a in enumerate(x) for b in
x[i+1:])
        middle=len(x)//2;xl,xr=x[:middle],x[middle:];cut=rank[xr[0][2]]
        yl=[p for p in y if rank[p[2]]<cut];yr=[p for p in y if rank[p[2]]>=cut]
        best=min(solve(xl,yl),solve(xr,yr));delta2=best[0]
        if delta2==0:return best
        ell=Fraction(xl[-1][0]+xr[0][0],2)
        z=[p for p in y if (p[0]-ell)**2<delta2]
        for i,a in enumerate(z):
            for b in z[i+1:i+8]:
                if (b[1]-a[1])**2>=best[0]:break
                best=min(best,(distance2(a,b),(a,b)))
        return best
    return solve(x,y)

if __name__=='__main__':
    print('hull',graham([(0,0),(4,0),(3,1),(4,4),(1,3),(0,4),(1,1)]))
    print('closest',closest([(0,0),(1,4),(2,1),(3,7),(4,1),(5,5),(6,0),(7,8)]))
```

原創驗算程式附錄: orientation_test.py

```
"""Use integer cross products to classify turns."""

def cross(origin, a, b):
    return (a[0] - origin[0]) * (b[1] - origin[1]) - (a[1] - origin[1]) * (b[0] - origin[0])


assert cross((0, 0), (2, 0), (1, 1)) > 0
assert cross((0, 0), (1, 2), (2, 2)) < 0
assert cross((0, 0), (1, 1), (2, 2)) == 0
print("left, right, collinear verified")
```

計算幾何詳解 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study